Introduction à l’Intelligence Artificielle

- **L’environnement**
  - Ouvrez Eclipse
  - Créez un nouveau projet Java ‘SOM’
  - Créez une classe **Main** et une classe **Neuron**

```
1
2
3  public class Main {
4
5
6      public Main(){
7
8      }
9
10     public static void main(String[] args) {
11
12         new Main();
13     }
14
15 }
16
```

Introduction à l'Intelligence Artificielle

- **L'environnement**
  - Téléchargez sur ***https://github.com/gaysimon/MachineLearning*** les classes DisplayFrame et DisplayPanel, et intégrez-les au projet
  - Ajoutez dans main une matrice de *Neuron* et un *DisplayFrame*, que vous initialiserez

```
2  public class Main {
3
4      private DisplayFrame display;       // display panel
5
6      public Neuron[][] SOM;
7
8      //////////////
9      public Main(){
10
11         SOM=new Neuron[20][20];
12         for (int i=0;i<20;i++){
13             for (int j=0;j<20;j++){
14                 SOM[i][j]=new Neuron(3);
15             }
16         }
17
18         display=new DisplayFrame(this);
```

Introduction à l’Intelligence Artificielle

- **Le neurone : plus simple que le neurone formel**
  - Dans la classe *Neuron* :
    - Un vecteur (tableau) de float pour les poids
    - Un float pour le résultat
  - Le constructeur de *Neuron* doit avoir le nombre de poids en paramètre
    - Initialisez le vecteur de poids avec ce nombre
    - Chaque poids doit être initialisé aléatoirement entre 0 et 1
      - Dans Main, vous ajouterez la valeur 1 au constructeur du neurone
  - Créez une fonction **`float compute(float[ ] img)`**
    - Implémentez la fonction qui calcule le résultat du neurone
    - La fonction doit enregistrer le résultat

Introduction à l'Intelligence Artificielle

- **Le neurone**

```
2  public class Neuron {
3
4      public float[] weights;
5      public float output;
6
7      ////////////////////////////////////////////
8      public Neuron(int size){
9          weights=new float[size];
10         for (int i=0;i<size;i++){
11                 weights[i]=(float)Math.random();
12         }
13     }
14
15     ////////////////////////////////////////////
16     public void compute(float[] img){
17         output=0;
18         for (int i=0;i<weights.length;i++){
19                 output+=(img[i]-weights[i])*(img[i]-weights[i]);
20         }
21         output=(float) Math.sqrt(output);
22     }
23
```

Introduction à l'Intelligence Artificielle

- **Le neurone**
  - Implémentez la fonction `void learn(float[] img, float ratio)`
    - Le ratio servira de coefficient de voisinage h
    - Ajoutez à la classe une variable learnRate initialisée à 0,02

$$w_i^{t+1} = w_i^t + \alpha \times h(r,k) \times (x_i - w_i^t)$$

Introduction à l'Intelligence Artificielle

- **Le neurone**

```
public class Neuron {

    public float learnRate=0.02f;
```

```
    ////////////////////////////////////////////
    public void learn(float[] img, float ratio){
        for (int i=0;i<weights.length;i++){
                weights[i]+= learnRate * ratio * (img[i]-weights[i]);
        }
    }
```

Introduction à l’Intelligence Artificielle

- **Le système d’entraînement**
  - Préparez un vecteur d’entrées input (type float, taille 3) et initialisez-le
  - Préparez une boucle de 100 000 itérations. Elle servira pour entraîner le réseau

```
7
8     public float[] input;
9
```

```
19    //////////////
20    public Main(){
21
22        // initialize structures
23        input=new float[3];
24
25        SOM=new Neuron[20][20];
26        for (int i=0;i<20;i++){
27            for (int j=0;j<20;j++){
28                SOM[i][j]=new Neuron(3);
29            }
30        }
31
32        // initialize display frame
33        display=new DisplayFrame(this);
34
35        for (int k=0;k<100000;k++){
36
37
38        }
```

Introduction à l'Intelligence Artificielle

- **Le système d'entraînement**
  - Chaque vecteur d'entraînement correspondra à une couleur aléatoire de l'espace RVB (Math.random())

```
for (int k=0;k<100000;k++){

    for (int c=0;c<3;c++) input[c]=(float) Math.random();
```

  - Calculez la valeur de sortie des neurones de la matrice

```
// compute neurons
for (int i=0;i<20;i++){
    for (int j=0;j<20;j++){
        SOM[i][j].compute(input);
    }
}
```

Introduction à l'Intelligence Artificielle

- **Le système d'entraînement**
  - Il faut déterminer quel neurone est le plus proche de l'exemple présenté.
    - Déterminez les coordonnées nx et ny du neurone avec la valeur minimale

```
    public int nx=0;
    public int ny=0;
    public float min=1;
```

```
            min=1;
            for (int i=0;i<20;i++){
                for (int j=0;j<20;j++){
                    if (SOM[i][j].output<min){
                        min=SOM[i][j].output;
                        nx=i;
                        ny=j;
                    }
                }
            }
```

Introduction à l'Intelligence Artificielle

- **Le système d'entraînement**
  - Nous allons nous occuper de la mise à jour des neurones
  - Créez la fonction de voisinage `float neighbor(float x)`
    - Nous utiliserons en paramètre le <u>carré de la distance</u>

$$h(r,k) = e^{\frac{-d(r,k)^2}{2\times\sigma^2}}$$

    - Utilisez $\sigma = 2$

```
public float neighbor(float x){
    return (float)Math.exp(-x/(8));
}
```

Introduction à l'Intelligence Artificielle

- **Le système d'entraînement**
  - Mettez à jour les neurones de la matrice avec la fonction

    ```
    public void learn(float[] img, float ratio)
    ```

  - Pour le ratio, on utilise la fonction de voisinage. Calculez d^2

```
for (int i=0;i<20;i++){
    for (int j=0;j<20;j++){
        SOM[i][j].learn(input, neighbor( (nx-i)*(nx-i)+(ny-j)*(ny-j)));
    }
}
```

Introduction à l'Intelligence Artificielle

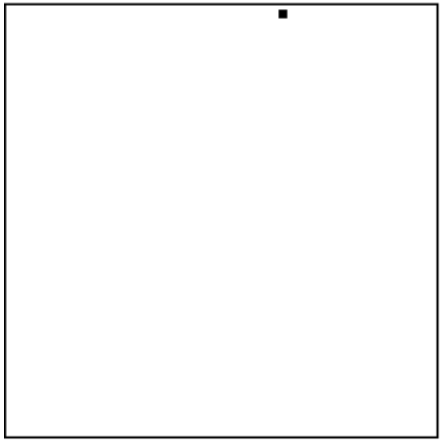
- **Le système d'affichage**
  - Ajoutez dans DisplayPanel.java l'affichage du vecteur d'entrées et la 'couleur' des neurones

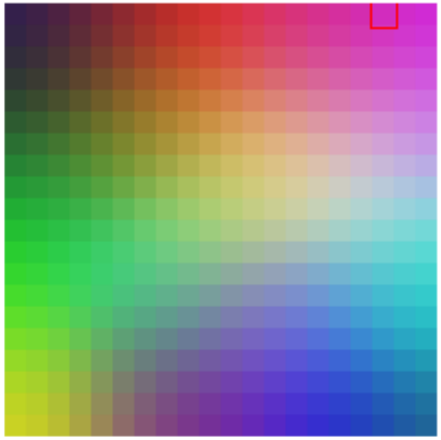
```
g.setColor(Color.black);
g.drawRect(0, 0, 200, 200);
g.fillOval(-2+(int)(main.input[0]*200), -2+(int)(main.input[1]*200), 5, 5);
```

```
float val1,val2,val3;
for (int i=0;i<20;i++){
    for (int j=0;j<20;j++){
        val1=main.SOM[i][j].weights[0];
        val2=main.SOM[i][j].weights[1];
        val3=main.SOM[i][j].weights[2];
        g.setColor(new Color(val1,val2,val3));
        g.fillRect(250+i*10, j*10, 10, 10);
    }
}
```

Introduction à l’Intelligence Artificielle

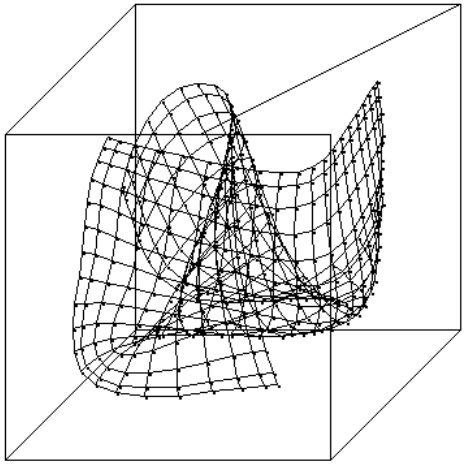
- **Testez votre système !**





  - Les neurones couvrent avec une matrice 2D un espace 3D de couleurs

Vous trouverez sur le github le code pour afficher la répartition des neurones en 3D



Introduction à l'Intelligence Artificielle

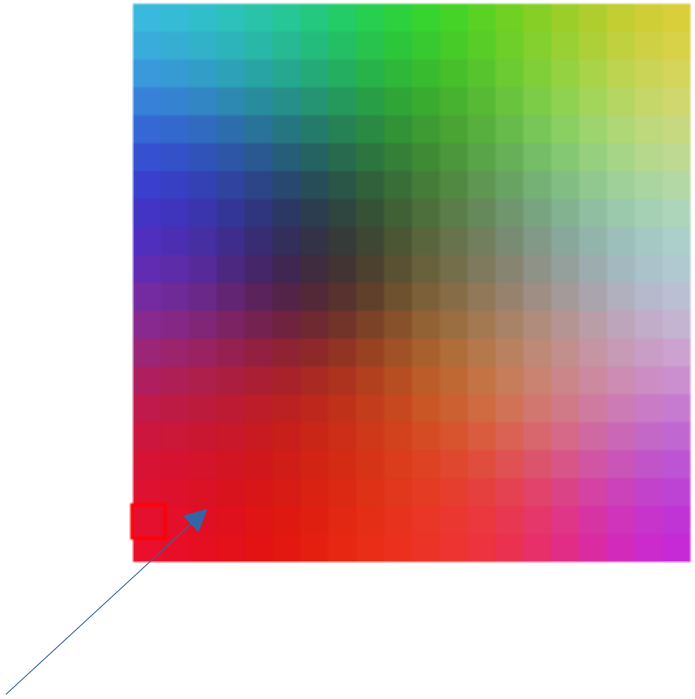
- **Amusons-nous un peu avec l'ensemble des couleurs**
  - Ajoutez le code suivant pour forcer une sur-représentation de certaines couleurs

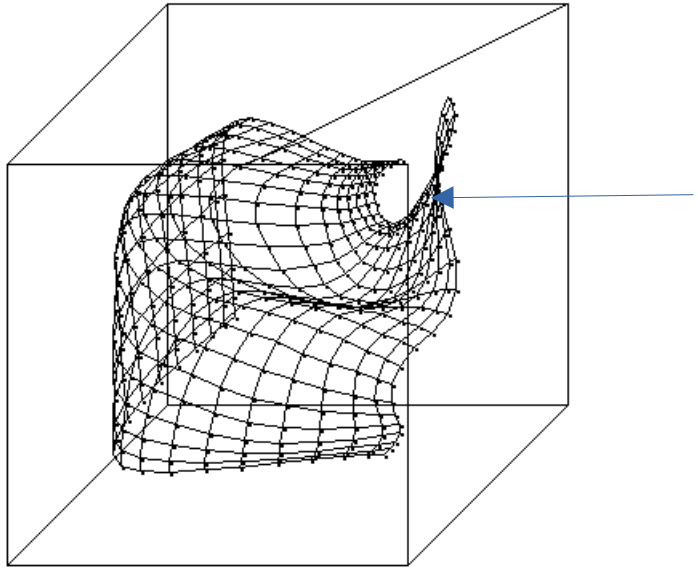
```
for (int c=0;c<3;c++) input[c]=(float) Math.random();

if (Math.random()<0.25){
    input[0]=input[0]/4+0.75f;  // red
    input[1]=input[1]/4;        // green
    input[2]=input[2]/4;        // blue
}
```

  - Vous pouvez tester avec d'autres couleurs

Introduction à l’Intelligence Artificielle

- **Amusons-nous un peu avec l’ensemble des couleurs**

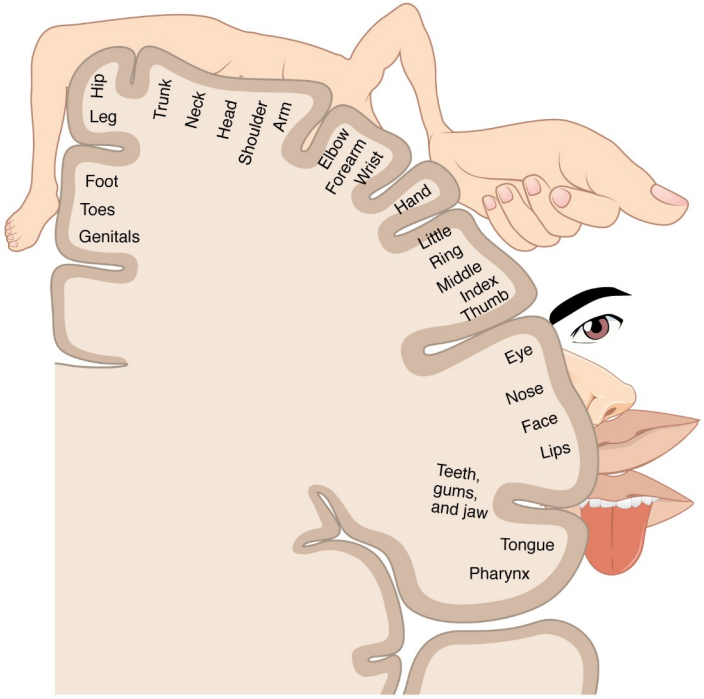




  - Les couleurs sont également sur-représentées sur la carte auto-adaptative

Introduction à l'Intelligence Artificielle

- **Cette propriété est aussi valable dans le cortex tactile !**

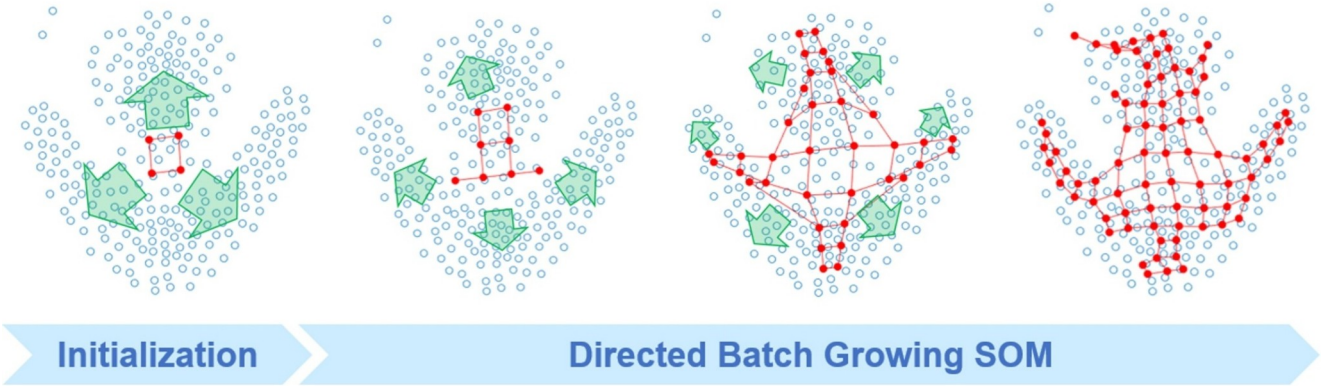


Homonculus de Penfield

- Même observations sur le cortex visuel (fovéa sur-représentée dans le cortex V1)

Introduction à l'Intelligence Artificielle

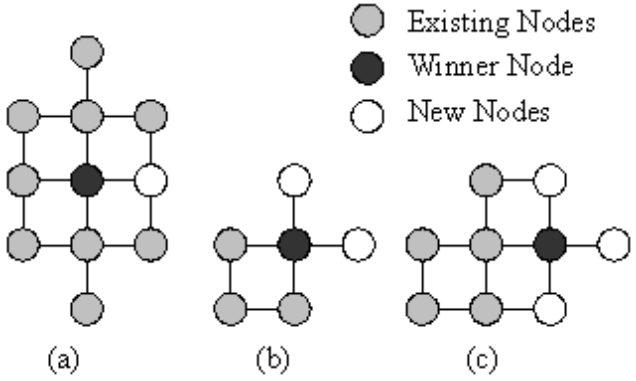
- **Variante du SOM : le Growing Self-Organizing Map**
  - Le nombre de neurone augmente progressivement
  - Le réseau s'étend dans la 'direction' ou se trouvent les données



Source : Mahdi Vasighi & HomaAmini, *A directed batch growing approach to enhance the topology preservation of self-organizing map, Applied Soft Computing, 2017*

Introduction à l'Intelligence Artificielle

- **Variante du SOM : le Growing Self-Organizing Map**
  - Le réseau commence avec un petit nombre de neurones (en général 4)
  - Si le neurone gagnant est en bordure de réseau, et que la distance est supérieure à un seuil prédéfini, des neurones voisins sont ajoutés au réseau



  - Le réseau va 'grandir' jusqu'à couvrir l'ensemble des valeurs d'entrées